Table 1: Security goals and the open source mechanisms that support them

Security goal	What it means	Typical open source mechanism
Authentication	Prove who a device or server is	X.509 certificates, pre-shared keys, mutual TLS
Confidentiality	Keep data unreadable to outsiders	TLS 1.3, DTLS 1.3, AES-GCM
Integrity	Detect any tampering with data	Authenticated encryption, digital signatures
Trusted firmware	Run only approved code	Signed images and secure boot (MCUboot)
Key management	Store and rotate secrets safely	Secret store in EdgeX, hardware key storage

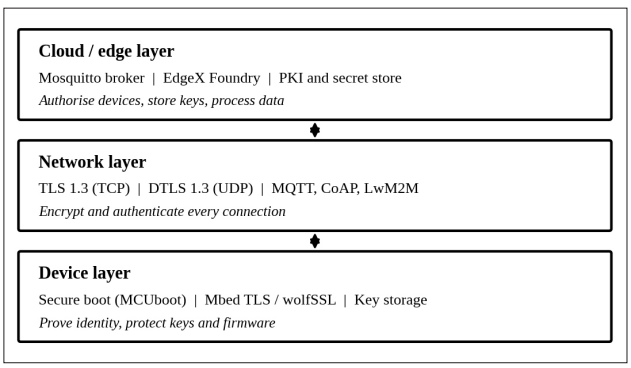


Figure 1: A layered view of IoT security and the open source tools commonly used at each layer (Source: Original diagram by the authors, based on the NIST baseline and EdgeX documentation. Reference: *https://docs.edgexfoundry.org/*)

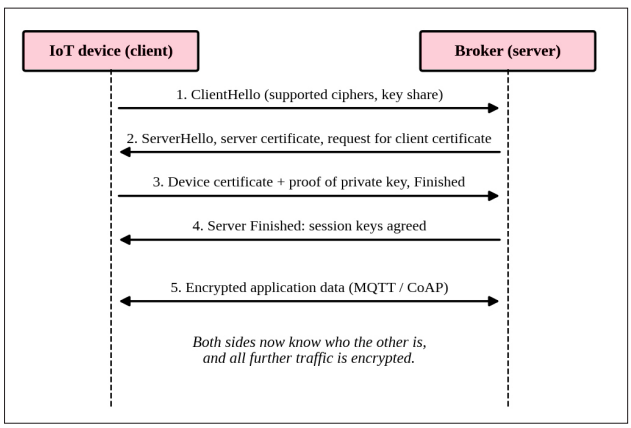


Figure 2: Simplified mutual TLS 1.3 handshake between an IoT device and a broker (Source: Redrawn by the authors from the handshake described in RFC 8446. Reference: *https://www.rfc-editor.org/rfc/rfc8446*)

Device authentication

Authentication gives every device an identity that cannot easily be copied. The simplest method is a pre-shared key (PSK), where the device and server hold the same secret. It is light on memory and processing power, but if one device is cracked, its key is exposed, and managing thousands of shared secrets gets messy fast.

The stronger and more common approach is certificate-based authentication. Each device gets its own private key and an X.509 certificate signed by a trusted certificate authority (CA). In mutual TLS, the server proves itself to the device and the device proves itself to the server, as sketched in Figure 2. Libraries such as Mbed TLS support this on very small devices. Table 2 compares the main options.

In practice, a message broker is the easiest place to enforce this. Eclipse Mosquitto, a popular open source MQTT broker, can be told to refuse any client that does not present a valid certificate. Listing 1 shows a short configuration that does exactly that.

Listing 1: Mosquitto configuration requiring client certificates

```
listener 8883
cafile /etc/mosquitto/certs/ca.crt
certfile /etc/mosquitto/certs/server.crt
keyfile /etc/mosquitto/certs/server.key
require_certificate true
use_identity_as_username true
allow_anonymous false
```

With these lines, the broker listens only on the encrypted port, rejects anonymous clients, and uses the name inside each device certificate as its username, so access rules can be written per device. The private key for each device should ideally live in a secure element or protected memory rather than in plain flash storage.

Table 2: Common authentication methods for IoT devices

Method	How it works	Strength	Weakness
Pre-shared key	Same secret stored on both sides	Small and fast	Hard to manage; one leak is costly
X.509 certificate	CA-signed identity per device	Strong, unique identity	Needs PKI and revocation
Raw public key	Public key known in advance	No CA needed; lightweight	Manual key distribution
Token (JWT, OAuth)	Short-lived signed token	Easy to expire and revoke	Needs a trusted token service
Hardware-backed key	Private key kept in a secure chip	Key cannot be copied out	Extra hardware cost